

REST

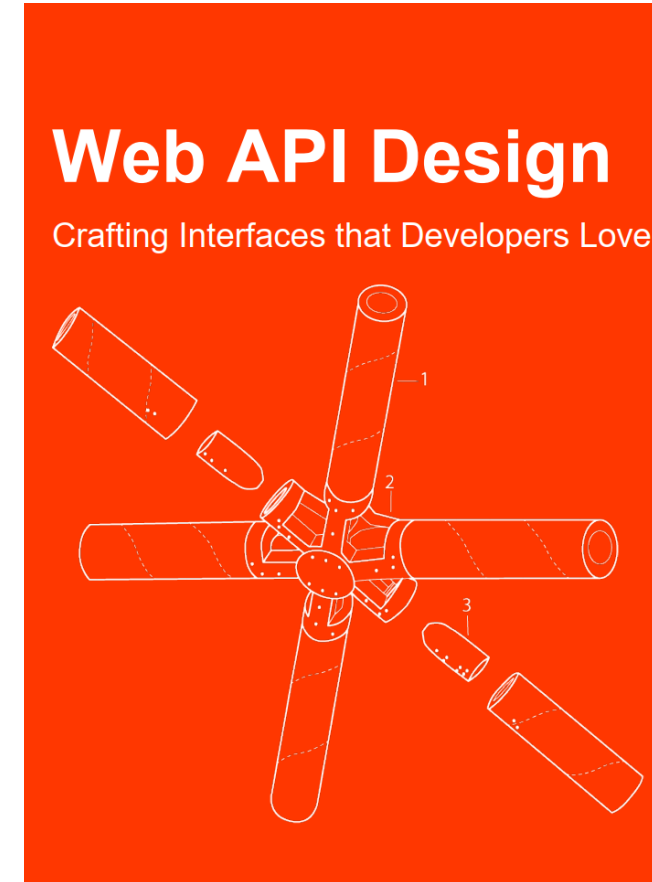
Representational State Transfer

Agenda

- Nouns are good; Verbs are bad
- Plural Nouns & Concrete Names
- Simplify associations & Sweep complexity under the ‘?’
- Handling errors
- Tips for versioning
- Pagination and partial response
- What about responses that don't involve resources?
- Supporting multiple formats
- Tips for search
- Richardson Maturity Model
- HATEOAS

Reference

- **Web API Design**
- Crafting Interfaces that Developers Love



- <https://pages.apigee.com/rs/apigee/images/api-design-ebook-2012-03.pdf>

Introduction

Introduction

- If you're reading this, chances are that you care about designing Web APIs that developers will love and that you're interested in applying proven design principles and best practices to your Web API.
- One of the sources for our design thinking is REST. Because REST is an architectural style and not a strict standard, it allows for a lot of flexibility. Because of that flexibility and freedom of structure, there is also a big appetite for design best practices.
- This e-book is ***a collection of design practices*** that we have developed in collaboration with some of the leading API teams around the world, as they craft their API strategy through a design workshop that we provide at Apigee.

Introduction

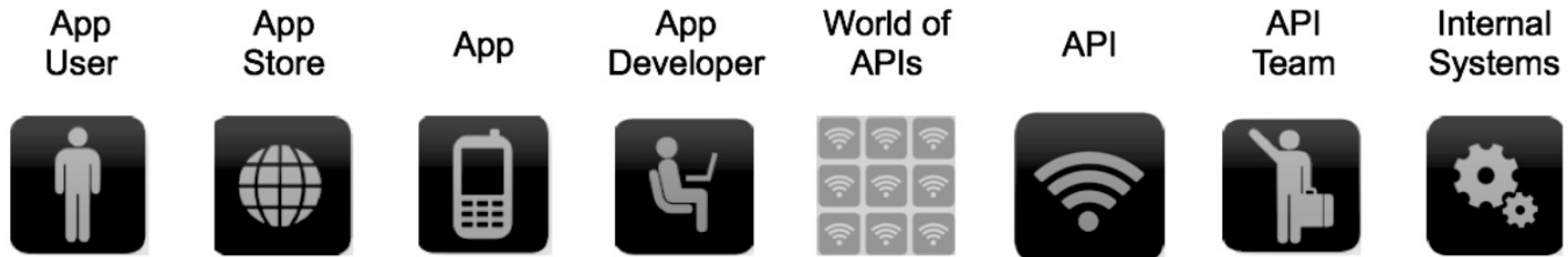
- We call our point of view in API design “pragmatic REST”, because it places the success of the developer over and above any other design principle.
- The developer is the customer for the Web API.
- The success of an API design is measured by how quickly developers can get up to speed and start enjoying success using your API.

Are you a Pragmatist or a RESTafarian?

- Approach API design from the ***‘outside-in’ perspective***.
- This means we start by asking
 - ***What are we trying to achieve with an API?***
- ***The API’s job is to make the application developer as successful as possible.***
- ***The orientation for APIs is to think about design choices from the application developer’s point of view.***

Are you a Pragmatist or a RESTafarian?

- *The primary design principle when crafting your API should be to maximize Application Developer productivity and success.*
- This is what we call **Pragmatic REST**.



Pragmatic REST is a design problem

- You have to get the design right, because *design communicates how something will be used*.
- The question becomes
 - What is the design with optimal benefit for the app developer?
- *The developer point of view is the guiding principle for all the specific tips and best practices, we've compiled.*

Nouns are good; verbs are bad

Keep your base URL simple and intuitive

- The number one principle in pragmatic RESTful design is: ***keep simple things simple***.
- ***Keep your base URL simple and intuitive***
- The base URL is the most important design affordance of your API.
- A simple and intuitive base URL design makes using your API easy.
- ***Affordance is a design property that communicates how something should be used without requiring documentation.***

Keep your base URL simple and intuitive

- A key litmus test we use for Web API design is that ***there should be only 2 base URLs per resource.***

- | | | |
|-------------------|---|---|
| /dogs | - | The first URL is for a collection |
| /dogs/1234 | - | The second is for a specific element in the collection |

- Boiling it down to this level will also force the verbs out of your base URLs.

Keep verbs out of your base URLs

- Many Web APIs start by using a method-driven approach to URL design.
- These method-based URLs sometimes contain verbs
 - sometimes at the beginning, sometimes at the end.
- For any resource that you model, like our dog, you can never consider one object in isolation.
- Rather, there are always related and interacting resources to account for
 - like owners, veterinarians, leashes, food, squirrels, and so on.

Keep verbs out of your base URLs

- Think about the method calls required to address all the objects in the dogs' world.
- The URLs for our resource might end up looking something like this.

```
...  
/getAllDogs  
/verifyLocation  
/feedNeeded  
/createRecurringWakeUp  
/giveDirectOrder  
/checkHealth  
/getRecurringWakeUpSchedule  
/getLocation  
/getDog
```

```
...  
/getAllLeashedDogs  
/verifyVeterinarianLocation  
/feedNeededFood  
/createRecurringMedication  
/doDirectOwnerDiscipline  
/doExpressCheckupWithVeterinarian  
/getRecurringFeedingSchedule  
/getHungerLevel  
/getSquirrelsChasingPuppies
```

- It's a slippery slope - soon you have a long list of URLs and no consistent pattern making it difficult for developers to learn how to use your API.

Use HTTP Methods Instead

- For our dog resources, we have two base URLs that use nouns as labels, and we can operate on them with HTTP verbs.
- Our HTTP verbs are **POST**, **GET**, **PUT**, and **DELETE**.
 - We think of them as mapping to the acronym, CRUD (Create-Read-Update-Delete)
- With our two resources (**/dogs** and **/dogs/1234**) and the four HTTP verbs, we have a rich set of capability that's intuitive to the developer.
- Here is a chart that shows what we mean for our dogs.

Use HTTP Methods Instead

Resource	POST	GET	PUT	DELETE
/dogs	Create a new dog	List All Dogs	Bulk Update Dogs	Delete All Dogs
/dogs/1234	Error	List that particular dog with id 1234	If exists: Update Else: Error	Delete that particular dog with id 1234

- The point is that developers probably don't need the chart to understand how the API behaves. They can experiment with and learn the API without the documentation.

Nouns are good; verbs are bad

- *In summary:*
 - Use two base URLs per resource.
 - Keep verbs out of your base URLs.
 - Use HTTP Methods to operate on the collections and elements.

Plural Nouns & Concrete Names

Plural Nouns

- Should you choose singular or plural nouns for your resource names?
 - You'll see popular APIs use both.
- Given that the first thing most people probably do with a RESTful API is a GET, we think ***it reads more easily and is more intuitive to use plural nouns.***
- But above all, ***avoid a mixed model*** in which you use singular for some resources, plural for others.
- ***Being consistent allows developers to predict and guess the method calls as they learn to work with your API.***

Concrete Names

- Achieving pure abstraction is sometimes a goal of API architects.
- However, that abstraction is not always meaningful for developers.
- Take for example an API that accesses content in various forms
 - blogs, videos, news articles, and so on.
- An API that models everything at the highest level of abstraction
 - as **/items** or **/assets** in our example - loses the opportunity to paint a tangible picture for developers to know what they can do with this API.
- It is more compelling and useful to see the resources listed
 - as **/blogs**, **/videos**, and **/articles**

Concrete Names

- The level of abstraction depends on your scenario.
 - You also want to expose a manageable number of resources.
 - Aim for concrete naming and to keep the number of resources between 12 and 24.
-
- *In summary,
an intuitive API uses **plural** rather than **singular** nouns,
and **concrete** rather than **abstract** names.*

Simplify Associations & Sweep Complexity under the ‘?’

Simplify Associations

- Resources almost always have relationships to other resources.
- What's a simple way to express these relationships in a Web API?
- Let's look again at our API, we had two base URLs: **/dogs** and **/dogs/1234**
- We're using HTTP verbs to operate on the resources and collections.
- ***Our dogs belong to owners.***
- To get all the dogs belonging to a specific owner, or to create a new dog for that owner, do a GET or a POST:
 - GET **/owners/5678/dogs**
 - POST **/owners/5678/dogs**

Simplify Associations

- Now, the *relationships can be complex*.
- Owners have relationships with veterinarians, who have relationships with dogs, who have relationships with food, and so on.
- It's not uncommon to see people string these together making a URL 5 or 6 levels deep.
- Remember that once you have the primary key for one level, you usually don't need to include the levels above because you've already got your specific object.
- In other words, you shouldn't need too many cases where a URL is deeper than what we have above */resource/identifier/resource*.

Sweep complexity behind the ‘?’

- Most APIs have intricacies beyond the base level of a resource.
- Complexities can include many states that can be updated, changed, queried, as well as the attributes associated with a resource.
- Make it simple for developers to use the base URL by putting optional states and attributes behind the HTTP question mark.
- To get all red dogs running in the park:
 - **GET /dogs?color=red&state=running&location=park**

Handling Errors

Why is good error design especially important for API designers?

- From the perspective of the developer consuming your Web API, everything at the other side of that interface is a black box.
- Errors therefore become a key tool providing context and visibility into how to use an API.
- **First**, developers learn to write code through errors, This is such an important and natural way for developers to work.
- **Secondly**, developers depend on well-designed errors at the critical times when they are troubleshooting and resolving issues.

How to think about errors in a pragmatic way with REST?

- **SimpleGeo**

```
HTTP Status Code: 401
{
  "code": 401,
  "message": "Authentication Required"
}
```

- SimpleGeo provides error codes but with no additional value in the payload.

How to think about errors in a pragmatic way with REST?

- **Facebook**

```
HTTP Status Code: 200
{
  "type": "OAuthException",
  "message": "(#803) Some of the aliases you requested do not exist: foo.bar"
}
```

- No matter what happens on a Facebook request, you get back the 200 - status code - everything is OK.
- Many error messages also push down into the HTTP response.
- Here they also throw an #803 error but with no information about what #803 is or how to react to it.

How to think about errors in a pragmatic way with REST?

- **Twilio**

```
HTTP Status Code: 401
{
  "status": "401",
  "message": "Authenticate",
  "code": 20003,
  "moreinfo": "http: //www.twilio.com/docs/errors/20003"
}
```

- They provide a more granular error message but with a link that takes you to the documentation.
- Community commenting and discussion on the documentation helps to build a body of information and adds context for developers experiencing these errors.

A couple of Best Practices

- Use HTTP status codes
- How many status codes should you use for your API?
- Make messages returned in the payload as verbose as possible.

Use HTTP status codes

- Use HTTP status codes and try to map them cleanly to relevant standard-based codes.
- There are over **70** HTTP Status Codes.
- However, most developers don't have all 70 memorized.
- So, if you choose status codes that are not very common you will force application developers away from building their apps and over to Wikipedia to figure out what you're trying to tell them.

Use HTTP status codes

- Therefore, most API providers use a small subset. For example, the Google GData API uses only 10 status codes; Netflix uses 9, and Digg, only 8.
- Google GData
 - **200 201 304 400 401 403 404 409 410 500**
- Netflix
 - **200 201 304 400 401 403 404 412 500**
- Digg
 - **200 400 401 403 404 410 500 503**

How many status codes should you use for your API?

- When you boil it down, there are really only **3** outcomes in the interaction between an app and an API:
 - Everything worked - **success**
 - The application did something wrong – **client error**
 - The API did something wrong – **server error**

How many status codes should you use for your API?

- Start by using the following 3 codes:
 - 200 - OK
 - 400 - Bad Request
 - 500 - Internal Server Error
- If you need more, try picking among these additional 5:
 - 201 - Created
 - 304 - Not Modified
 - 404 - Not Found
 - 401 - Unauthorized
 - 403 - Forbidden
- But you shouldn't need to go beyond 8.

Make messages returned in the payload as verbose as possible

- **Code for Code**

- 200 – OK
- 401 – Unauthorized

- **Message for People**

```
{  
  "developerMessage": "Verbose, plain language description of the problem  
                        for the app developer with hints about how to fix it.",  
  "userMessage": "Pass this message on to the app user if needed.",  
  "errorCode": 12345,  
  "more info": "http://dev.teachdogrest.com/errors/12345"  
}
```

- In summary, be verbose and use plain language descriptions.
- Add as many hints as your API team can think of about what's causing an error.
- We highly recommend you add a link in your description to more information.

Tips for Versioning

Tips for versioning

- Versioning is one of the most important considerations when designing your Web API.
- ***Never release an API without a version and make the version mandatory.***
- Let's see how three top API providers handle versioning.
 - Twilio `/2010-04-01/Accounts/`
 - salesforce `/services/data/v20.0/subjects/Account`
 - Facebook `?v=1.0`

Complex
Frequent Change
WTH

How to think about version numbers in a pragmatic way with REST?

- Never release an API without a version.
- Make the version mandatory.
- Specify the version with a '**v**' prefix.
- Move it all the way to the left in the URL so that it has the highest scope e.g. (**/v1/dogs**).
- Use a simple ordinal number.
- Don't use the dot notation like v1.2 because it implies a granularity of versioning that doesn't work well with APIs, it's an interface not an implementation.
- Stick with **v1**, **v2**, and so on.

How to think about version numbers in a pragmatic way with REST?

- How many versions should you maintain?
 - ***Maintain at least one version back.***
- For how long should you maintain a version?
 - ***Give developers at least one cycle to react before obsoleting a version.***
- Sometimes that's 6 months; sometimes it's 2 years.
- It depends on your developers' development platform, application type, and application users.
- For example, mobile apps take longer to rev' than Web apps.

Should version and format be in URLs or headers?

- There is a strong school of thought about putting format and version in the header.
- Using headers is more correct for many reasons: it leverages existing HTTP standards, it's intellectually consistent with Fielding's vision, it solves some hard real-world problems related to inter-dependent APIs, and more.
- However, we think the reason most of the popular APIs do not use it is because it's less fun to hack in a browser.

Should version and format be in URLs or headers?

- Simple rules we follow:
 - *If it changes the logic you write to handle the response, put it in the URL so you can see it easily.*
 - *If it doesn't change the logic for each response, like OAuth information, put it in the header.*

Should version and format be in URLs or headers?

- These for example, all represent the same resource:
 - /dogs/1
 - Content-Type: application/json
 - /dogs/1
 - Content-Type: application/xml
 - /dogs/1
 - Content-Type: application/png
- The code we would write to handle the responses would be very different.
- There's no question the header is more correct, and it is still a very strong API design.

Pagination & Partial Response

Partial Response

- Partial response allows you to give developers just the information they need.
- Take for example a request for a tweet on the Twitter API. You'll get much more than a typical twitter app often needs - including the name of person, the text of the tweet, a timestamp, how often the message was re-tweeted, and a lot of metadata.
- Let's look at how several leading APIs handle giving developers just what they need in responses, including Google who pioneered the idea of partial response.

Partial Response

- ***LinkedIn***

- /people:(id,first-name,last-name,industry)
- This request on a person returns the ID, first name, last name, and the industry.
- LinkedIn does partial selection using this terse :(...) syntax which isn't self-evident.
- Plus, it's difficult for a developer to reverse engineer the meaning using a search engine.

- ***Facebook***

- /joe.smith/friends?fields=id,name,picture

- ***Google***

- ?fields=title,media:group(media:thumbnail)

- Google and Facebook have a similar approach, which works well.
- They each have an optional parameter called ***fields*** after which you put the names of fields you want to be returned.

Add optional fields in a comma-delimited list

- The Google approach works extremely well.
- Here's how to get just the information we need from our dogs API using this approach:
 - `/dogs?fields=name,color,location`
- It's simple to read; a developer can select just the information an app needs at a given time; it cuts down on bandwidth issues, which is important for mobile apps.
- The partial selection syntax can also be used to include associated resources cutting down on the number of requests needed to get the required information.

Pagination

- It's almost always a bad idea to return every resource in a database.
- Let's look at how Facebook, Twitter, and LinkedIn handle pagination.
 - Facebook uses offset and limit.
 - Twitter uses page and rpp (records per page).
 - LinkedIn uses start and count
- Semantically, Facebook and LinkedIn do the same thing.
 - That is, the LinkedIn start & count is used in the same way as the Facebook offset & limit.
- To get records 50 through 75 from each system, you would use:
 - Facebook - offset 50 and limit 25
 - Twitter - page 3 and rpp 25 (records per page)
 - LinkedIn - start 50 and count 25

Use limit and offset

- We recommend limit and offset.
- It is more common, well understood in leading databases, and easy for developers.
 - `/dogs?limit=25&offset=50`
- Metadata
 - We also suggest including metadata with each response that is paginated that indicated to the developer the total number of records available.
- What about defaults?
 - My loose rule of thumb for default pagination is `limit=10` with `offset=0`. (`limit=10&offset=0`)
- The pagination defaults are of course dependent on your data size.
- If your resources are large, you probably want to limit it to fewer than 10; if resources are small, it can make sense to choose a larger limit.

Pagination & Partial Response

- **In summary:**
 - Support partial response by adding optional fields in a comma delimited list.
 - Use limit and offset to make it easy for developers to paginate objects.

What about Responses
that don't involve resources?

Responses that don't involve resources

- API calls that send a response that's not a resource per se are not uncommon depending on the domain.
- Actions like the following:
 - /calculate
 - /translate
 - /convert
- For example, you want to make a simple algorithmic calculation like how much tax someone should pay **or** do a natural language translation (one language in request; another in response) **or** convert one currency to another.
- ***None involve resources returned from a database.***

Responses that don't involve resources

- In these cases:
- ***Use verbs not nouns***
 - For example, an API to convert 100 euros to Chinese Yen:
 - `/convert?from=EUR&to=CNY&amount=100`
- ***Make it clear in your API documentation that these “non-resource” scenarios are different.***
 - Simply separate out a section of documentation that makes it clear that you use verbs in cases like this – where some action is taken to generate or calculate the response, rather than returning a resource directly.

Supporting Multiple Formats

Supporting Multiple Formats

- We recommend that you support more than one format – that you push things out in one format and accept as many formats as necessary.
- Here's what the syntax looks like for a few key APIs.
 - Google Data
 - `?alt=json`
 - Foursquare
 - `/venue.json`
 - Digg*
 - `Accept: application/json`
 - `?type=json`
 - * The type argument, if present, overrides the Accept header.
- Digg allows you to specify in two ways: in a pure RESTful way in the Accept header or in the type parameter in the URL. This can be confusing - at the very least you need to document what to do if there are conflicts.

Supporting Multiple Formats

- We recommend the Foursquare approach.
- To get the JSON format from a collection or specific element:
 - `dogs.json`
 - `/dogs/1234.json`
- Developers and even casual users of any file system are familiar to this dot notation.
- It also requires just one additional character (the period) to get the point across.
- What about default formats?
 - In my opinion, JSON is winning out as the default format.
 - JSON is the closest thing we have to universal language.
 - Even if the back end is built in Ruby on Rails, PHP, Java, Python etc., most projects probably touch JavaScript for the front-end.
 - It also has the advantage of being terse - less verbose than XML.

What about attribute names?

- Here are API responses from a few leading APIs:
 - Twitter
 - "created_at": "Thu Nov 03 05:19:38 +0000 2011"
 - Bing
 - "DateTime": "2011-10-29T09:35:00Z"
 - Foursquare
 - "createdAt": 1320296464
- They each use a different code convention.
- We think that Foursquare has the best approach (***camel case***).

What about attribute names?

- How does the API response get back in the code?
- You parse the response (JSON parser); what comes back populates the Object.
- It looks like this
 - `var myObject = JSON.parse(response);`
- If you chose the Twitter or Bing approach, your code looks like this.
 - `timing = myObject.created_at;`
 - `timing - myObject.DateTime;`
- Its not JavaScript convention and looks weird – looks like the name of another object or class in the system, which is not correct.

Recommendations

- Use JSON as default
- Follow JavaScript conventions for naming attributes
 - Use medial capitalization (aka CamelCase)
 - Use uppercase or lowercase depending on type of object
- This results in code that looks like the following, allowing the JavaScript developer to write it in a way that makes sense for JavaScript.
 - `"createdAt": 1320296464`
 - `timing = myObject.createdAt;`

Tips for Search

Tips for search

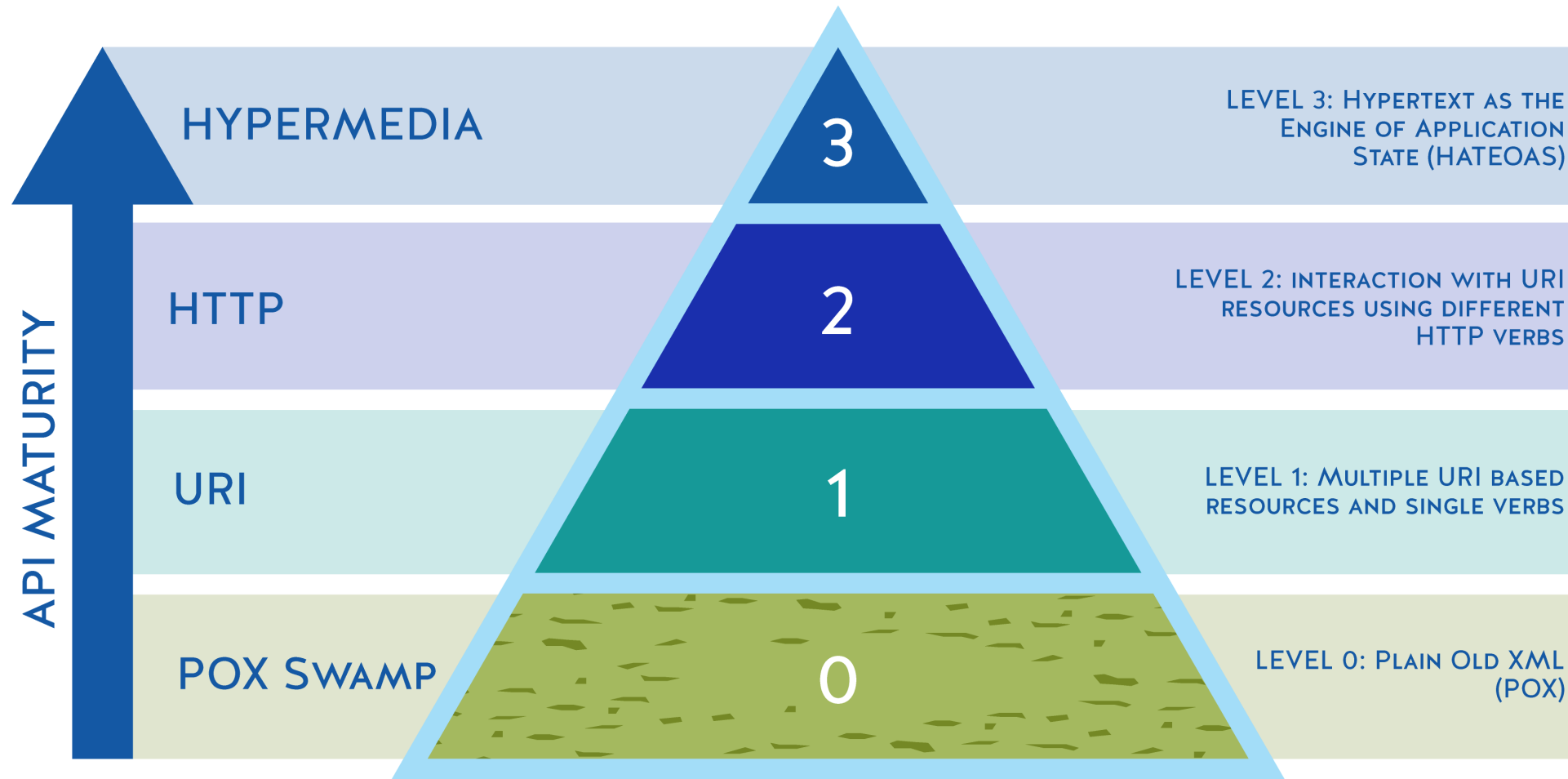
- While a simple search could be modeled as a resourceful API (for example, **dogs/?q=red**), a more complex search across multiple resources requires a different design.
- This will sound familiar if you've read the topic about using verbs not nouns when results don't return a resource from the database - rather the result is some action or calculation.
- If you want to do a global search across resources, we suggest you follow the Google model:

The Google model

- Global Search:
 - **/search?q=fluffy+fur**
 - Here, search is the verb; **?q** represents the query.
- Scoped search:
 - **/owners/5678/dogs?q=fluffy+fur**
 - Search in dogs owned by resource ID 5678
 - Notice that we've dropped the explicit search in the URL and rely on the parameter 'q' to indicate the scoped query.
- Formatted results
 - **/search.xml?q=fluffy+fur**
 - For search or for any of the action oriented (non-resource) responses, you can prepend with the format.

Richardson Maturity Model

Richardson Maturity Model



HATEOAS

HATEOAS

- HATEOAS: **H**ypermedia **A**s **T**he **E**ngine **O**f **A**pplication **S**tate
 - HATEOAS is a component of the REST application architecture that distinguishes it from other network application architectures.
- With HATEOAS, a client interacts with a network application whose application servers provide information dynamically through *hypermedia*.
- A REST client needs little to no prior knowledge about how to interact with an application or server beyond a generic understanding of hypermedia.
- The restrictions imposed by HATEOAS decouple client and server.
- This enables server functionality to evolve independently.
- Ideally, a RESTful API that supports HATEOAS doesn't Need any Documentation.

Example

```
GET /accounts/12345 HTTP/1.1
Host: bank.example.com
Accept: application/json
...
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: ...

{
  "account": {
    "account_number": 12345,
    "balance": {
      "currency": "usd",
      "value": 100.00
    },
    "links": {
      "deposit": "/accounts/12345/deposit",
      "withdraw": "/accounts/12345/withdraw",
      "transfer": "/accounts/12345/transfer",
      "close": "/accounts/12345/close"
    }
  }
}
```